

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Analiza Arhitecturală a Microserviciului **Statistici**

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Introducere și Contextul Sistemului	2
1.1	Harta de Context (Context Map)	2
2	Arhitectura Internă a Microserviciului Statistici	2
2.1	Diagrama de Pachete (Arhitectura Hexagonală)	2
2.2	Diagrama de Clase Detaliată	3
3	Integrarea Datelor și Sursa Unică de Adevăr	4
4	Modelarea Domeniului (Domain-Driven Design)	4
4.1	Aggregate Root — Clasa <code>StatisticaGrafic</code>	5
4.2	Entități Locale (Data Transfer Models)	5
5	Șabloane de Proiectare (Design Patterns) Utilizate	6
5.1	Șablonul Adaptor (Adapter / Wrapper)	6
5.2	Șablonul Singleton (Gestionat prin IoC)	6
6	Concluzii	6

1 Introducere și Contextul Sistemului

Proiectul curent implementează un sistem backend complex pentru o agenție de bilete de avion. Pentru a asigura scalabilitatea, mentenabilitatea și independența echipelor de dezvoltare, arhitectura globală a fost divizată în **Microservicii**, ghidate de limitele contextuale (*Bounded Contexts*) din Domain-Driven Design (DDD).

1.1 Harta de Context (Context Map)

Sistemul global este împărțit în șase contexte principale. Microserviciul **Statistici** acționează ca un agregator de date și este un consumator direct (*Downstream*) pentru contextele **Zboruri** și **Bilete**.

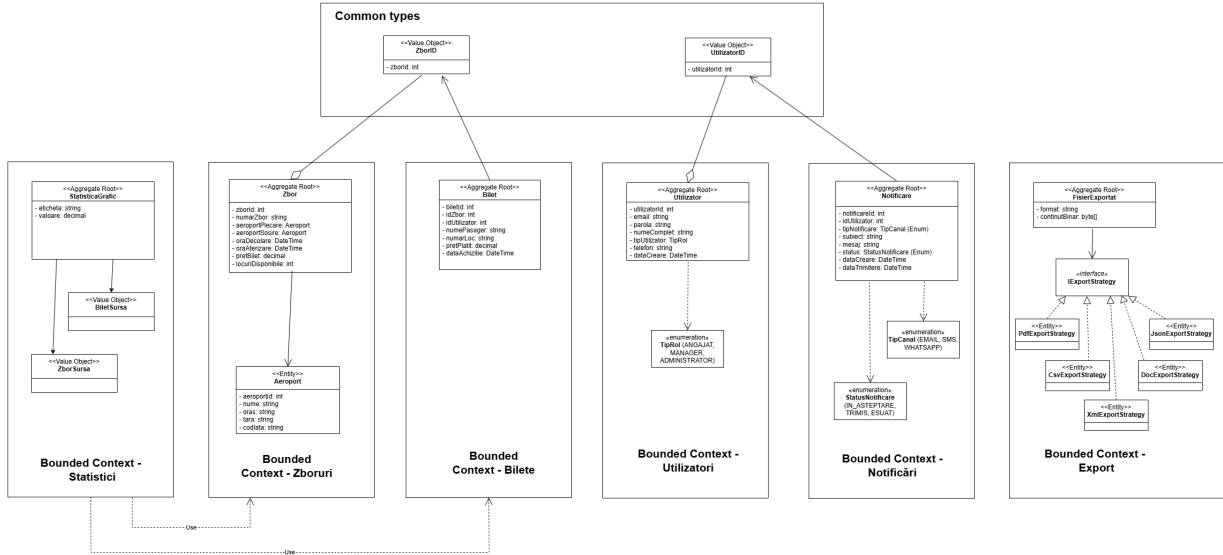


Figura 1: Harta de context a sistemului global de gestiune

Justificarea deciziei: Această decizie arhitecturală previne crearea unui monolit rigid și a unui punct unic de eșec (Single Point of Failure). Contextul **Statistici** primește independență totală de baze de date centralizate; rolul său este pur analitic. Entitățile comune, precum identificatorii zborurilor, sunt utilizate pentru a corela datele din mai multe surse (zboruri și rezervări) folosind referințe, protejând intimitatea și structura serviciilor sursă.

2 Arhitectura Internă a Microserviciului Statistici

2.1 Diagrama de Pachete (Arhitectura Hexagonală)

La nivel intern, microserviciul **Statistici** respectă cu strictețe principiul Inversiunii Dependențelor (*Dependency Inversion Principle*), specific Arhitecturii Hexagonale (sau Ports and Adapters).

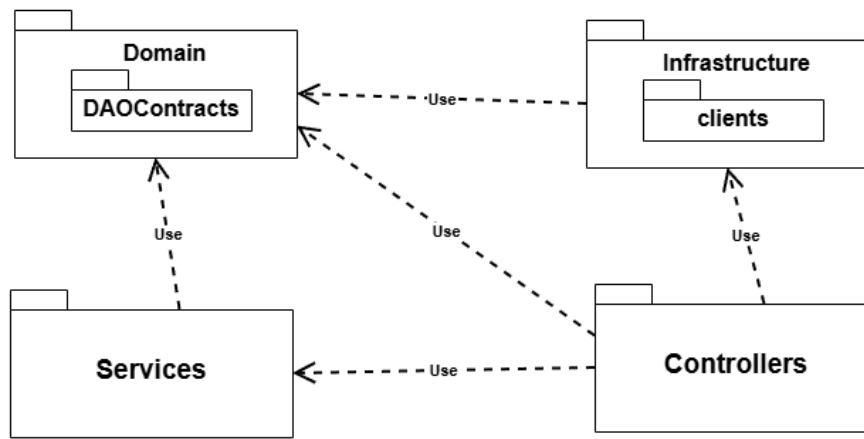


Figura 2: Diagrama de pachete: Organizarea internă pe straturi

Justificarea deciziei: Săgețile din figura 2 indică direcția dependențelor (-Use-). Se observă clar că inima aplicației este pachetul **Domain**, care nu importă elemente din straturile inferioare sau superioare. Pachetul **Infrastructure** (conținând detaliile tehnice de integrare HTTP cu serviciile externe) depinde de **Domain** pentru a-i implementa interfețele (**DAOContracts**). Pachetul **Controllers** apelează pachetul **Services**, care conține regulile de business izolate.

2.2 Diagrama de Clase Detaliată

Pentru a expune în mod granular structura internă a modulelor, tipurile de date, precum și comportamentele metodelor din fiecare strat, a fost mapată structura de clase aferentă acestui microserviciu.

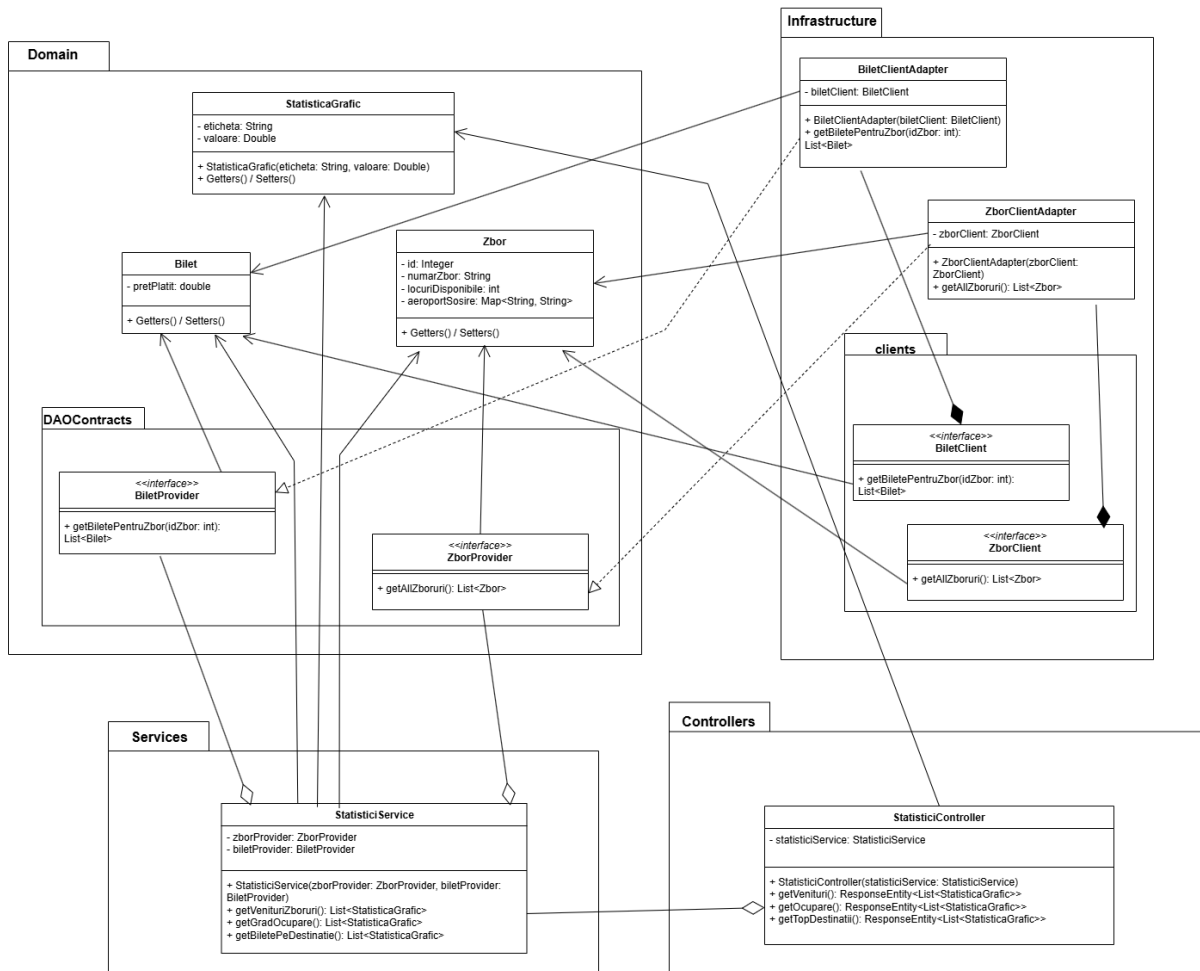


Figura 3: Diagrama de clase detaliată a microserviciului Statistici

Justificarea deciziei: Figura 3 oferă o vedere structurală completă a mecanismului de decuplare. Ea evidențiază injectarea porturilor de date (**ZborProvider**, **BiletProvider**) direct în constructorul clasei **StatisticiService** folosind asocierea de agregare, în timp ce realizarea acestor contracte se face strict la nivelul infrastructurii (**ZborClientAdapter**, **BiletClientAdapter**), încapsulând astfel clienții de rețea.

3 Integrarea Datelor și Sursa Unică de Adevăr

Spre deosebire de o arhitectură tradițională ce se bazează pe o bază de date relațională proprie, microserviciul **Statistici** funcționează într-un regim *stateless* (fără memorie de durată proprie).

Justificarea deciziei: Deoarece acest serviciu analizează date pe care nu le controlează direct (aparținând serviciilor **Zboruri** și **Bilete**), s-a optat pentru comunicarea *inter-process* prin HTTP. Pachetul *Infrastructure* conține clienți Feign care interoghează resursele pe rețea. Această alegere impune respectarea principiului Single Source of Truth: evităm duplicarea zborurilor în mai multe baze de date (și implicit problemele de sincronizare). Calculele se fac pe seturi de date actualizate în timp real de la microserviciile care le dețin.

4 Modelarea Domeniului (Domain-Driven Design)

Nucleul logicii de analiză este izolat conceptual în **Domain**. Modelele reflectă doar datele esențiale necesare.

4.1 Aggregate Root — Clasa StatisticaGrafic

Clasa `StatisticaGrafic` acționează ca punctul de agregare final (Aggregate Root) pentru prezentare. Ea încapsulează un format uniform (cuprinzând o `eticheta` și o `valoare` generică) care poate acomoda rapoarte variate: venituri monetare, procente de ocupare sau clasamente ale destinațiilor populare, simplificând munca aplicației frontend de afișare a dashboard-urilor.

4.2 Entități Locale (Data Transfer Models)

În mod intenționat, modelele de `Zbor` și `Bilet` definite în acest Domeniu nu reprezintă copii identice ale tabelelor din serviciile originale. Ele expun o structură redusă, acționând ca entități/obiecte de transfer locale, conținând doar proprietățile matematice vizate: `pretPlatit`, `locuriDisponibile` și identificatorii locațiilor (`aeroportSosire`).

5 Șabloane de Proiectare (Design Patterns) Utilizate

5.1 Șablonul Adaptor (Adapter / Wrapper)

Pentru a separa complet inima microserviciului de modalitățile tehnice prin care sunt preluate informațiile (în cazul nostru HTTP via Spring Cloud OpenFeign), a fost implementat șablonul architectural Adaptor.

Explicație și Justificare: În cadrul arhitecturii ilustrate în Figura 3, interfețele `ZborProvider` și `BiletProvider` reprezintă **Porturile** abstracte definite de *DAOContracts*. Clasele `ZborClientAdapter` și `BiletClientAdapter` sunt **Adaptoarele** specifice din Infrastructură. Aceste adaptoare primesc datele tehnice prin clienții Feign și le traduc direct sub forma contractelor dictate de porturi. Astfel, Serviciul de calcule (`StatisticiService`) rămâne neafectat de erorile HTTP, parsing-ul JSON sau eventuale tranziții de protocoale (ex: de la REST la gRPC).

5.2 Șablonul Singleton (Gestionat prin IoC)

Singleton este șablonul folosit pentru a instanția o singură dată entitățile orchestratoare, prevenind risipa de memorie și procesor la solicitările masive de generare de rapoarte din partea administratorilor.

Explicație și Justificare: Această funcționalitate este susținută de containerul *Inversion of Control (IoC)* oferit de framework-ul Spring. Clasele anotate cu `@RestController` (`StatisticiController`), `@Service` (`StatisticiService`) și `@Component` (`BiletClientAdapter`) sunt recunoscute și construite automat într-o instanță unică (Singleton Bean). Deoarece prelucrările de date se fac exclusiv pe variabile locale în interiorul funcțiilor specifice (*stateless processing*), toate cerințele concurente venite din Frontend pot accesa simultan și în siguranță (thread-safe) aceeași instanță a serviciului matematic.

6 Concluzii

Microserviciul **Statistici** reflectă aplicarea matură a principiilor SOLID și a Arhitecturii Clean/Hexagonale. Prin izolarea logicii matematice în Domain, comunicarea descentralizată asigurată de pattern-ul Adaptor și eficiența delegată containerului de injecție a dependențelor, se obține o componentă scalabilă, ușor de testat și independentă tehnologic, adaptată excelent ecosistemului de distribuție al agenției.